

DS4023 (2024 Fall)
Machine Learning
Department of Computer Science
BNU-HKBU United International College

Non-linear manifold embedding



Chapter 5: Dimensionality reduction

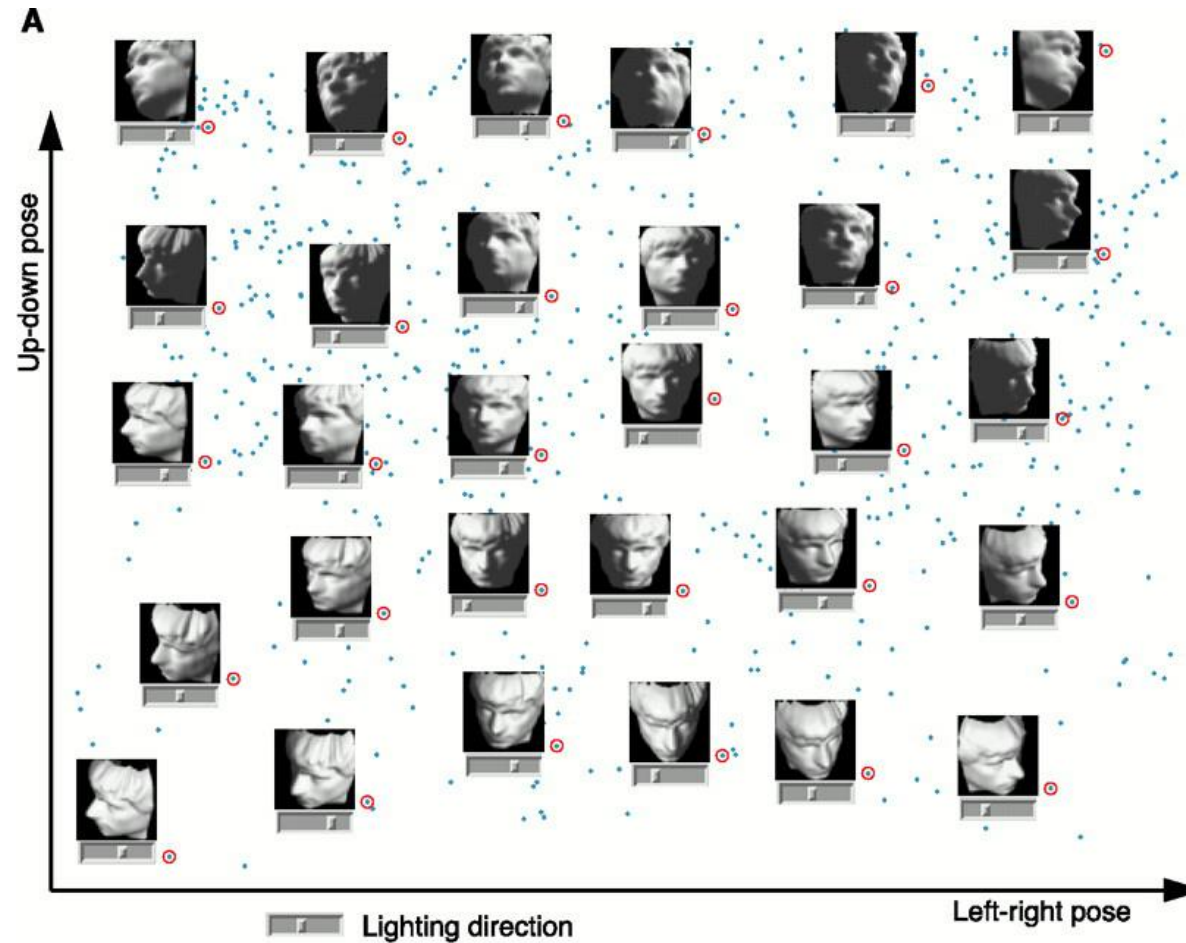
Dimensionality reduction

Objective: to find a small number of features that represent a large number of observed dimensions.



For each image: there are $64 \times 64 = 4096$ pixels (observed dimensions)

Dimensionality Reduction



Reduced to 3-dimensional manifold: two pose variables + azimuthal lighting angle

Dimensionality reduction: Some Assumptions

- ▶ High-dimensional data often lies on or near a much lower dimensional, curved manifold.
- ▶ A good way to represent data points is by their low-dimensional coordinates.
- ▶ The low-dimensional representation of the data should capture information about high-dimensional pairwise distances.

The basic idea of non-parameteric dimensionality reduction

- ▶ Represent each data-point by a point in a lower dimensional space.
- ▶ Choose the low-dimensional points so that they optimally represent some property of the data-points (e.g. the pairwise distances).
 - ▶ Many different properties have been tried.
- ▶ Do not insist on learning a parametric “encoding” function that maps each individual data-point to its low-dimensional representative.
- ▶ Do not insist on learning a parametric “decoding” function that reconstructs a data-point from its low dimensional representative.

Two types of dimensionality reduction

- ▶ **Global methods** assume that all pairwise distances are of equal importance.
 - ▶ Choose the low-D pairwise distances to fit the high-D ones (using magnitude or rank order).
- ▶ **Local methods** assume that only the local distances are reliable in high-D.
 - ▶ Put more weight on modeling the local distances correctly.

Linear methods of reducing dimensionality

- ▶ **PCA** finds the directions that have the most variance.
 - ▶ By representing where each datapoint is along these axes, we minimize the squared reconstruction error.
 - ▶ Linear autoencoders are equivalent to PCA
- ▶ **Multi-Dimensional Scaling** arranges the low-dimensional points so as to minimize the discrepancy between the pairwise distances in the original space and the pairwise distances in the low-D space.

Metric Multi-Dimensional Scaling

- ▶ Find low dimensional representatives, y , for the high-dimensional data-points, x , that preserve pairwise distances as well as possible.
- ▶ An obvious approach is to start with random vectors for the y 's and then perform steepest descent by following the gradient of the cost function.
- ▶ Since we are minimizing squared errors, maybe this has something to do with PCA?
 - ▶ If so, we don't need an iterative method to find the best embedding.

$$Cost = \sum_{i < j} (d_{ij} - \hat{d}_{ij})^2$$

$$d_{ij} = \|x_i - x_j\|^2$$

$$\hat{d}_{ij} = \|y_i - y_j\|^2$$

Converting metric MDS to PCA

- ▶ If the data-points all lie on a hyperplane, their pairwise distances are perfectly preserved by projecting the high-dimensional coordinates onto the hyperplane.
 - ▶ So in that particular case, PCA is the right solution.
- ▶ If we “double-center” the data, metric MDS is equivalent to PCA.
 - ▶ Double centering means making the mean value of every row and column be zero.
 - ▶ But double centering can introduce spurious structure.

Other non-linear methods of reducing dimensionality

- ▶ Non-linear autoencoders with extra layers are much more powerful than PCA but they can be slow to optimize and they get different, locally optimal solutions each time.
- ▶ Multi-Dimensional Scaling can be made non-linear by putting more importance on the small distances. A popular version is the Sammon mapping:

$$Cost = \sum_{i,j} \left(\frac{\overset{\text{high-D distance}}{\downarrow} \|\mathbf{x}_i - \mathbf{x}_j\| - \overset{\text{low-D distance}}{\downarrow} \|\mathbf{y}_i - \mathbf{y}_j\|}{\|\mathbf{x}_i - \mathbf{x}_j\|} \right)^2$$

- ▶ Non-linear MDS is also slow to optimize and also gets stuck in different local optima each time.

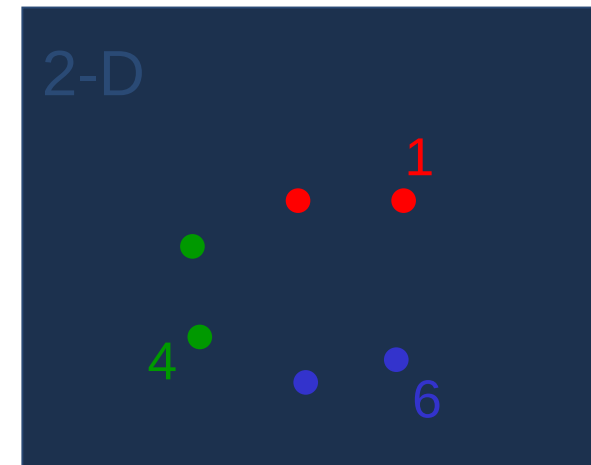
Problems with Sammon mapping

- ▶ It puts too much emphasis on getting very small distances exactly right.
- ▶ It produces embeddings that are circular with roughly uniform density of the map points.

IsoMap: Local MDS without local optima

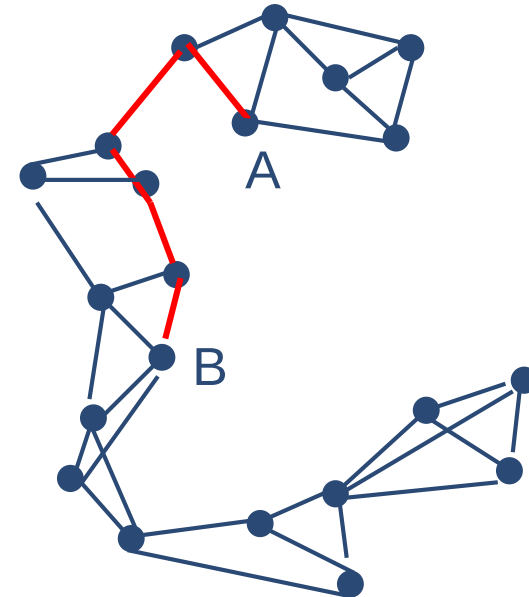
- ▶ Instead of only modeling local distances, we can try to measure the distances along the manifold and then model these intrinsic distances.
- ▶ The main problem is to find a robust way of measuring distances along the manifold.
- ▶ If we can measure manifold distances, the global optimisation is easy: It's just global MDS (i.e. PCA)

If we measure distances along the manifold,
 $d(\textcolor{red}{1}, \textcolor{blue}{6}) > d(\textcolor{red}{1}, \textcolor{green}{4})$



How Isomap measures intrinsic distances

- ▶ Connect each datapoint to its K nearest neighbors in the high-dimensional space.
- ▶ Put the true Euclidean distance on each of these links.
- ▶ Then approximate the manifold distance between any pair of points as the shortest path in this “neighborhood graph”.



Isometric Feature Mapping (Isomap)

- A nonlinear method for dimensionality reduction
- Finds the map that preserves the global, nonlinear geometry of the data by preserving the geodesic manifold interpoint distances
- Geodesic: Shortest curve along the manifold connecting two points
- First approximates the geodesic interpoint distances, then runs MDS to find the projection that preserves these distances

2 components:

- How do we measure the geodesic distances on the manifold?
- How to map points on the Euclidean space in lower dimensional space?

Multidimensional Scaling (MDS)

The algorithm detects meaningful underlying dimensions that explain observed similarities or dissimilarities (distances) between the investigated objects.

Given:

$n \times n$ matrix of dissimilarities between n objects

$$(\Delta^T = \Delta; \delta_{ij} \geq 0; \delta_{ii} = 0) \quad d$$

OR $n \times n$ matrix of similarities between n objects

$$(\Theta^T = \Theta; \theta_{ij} \leq \theta_{ii})$$

convert to dissimilarities $\rightarrow \delta_{ij}^2 = \theta_{ii} + \theta_{jj} - 2\theta_{ij}$

Goal: Find a configuration in a low-dimensional Euclidean space R^k whose interpoint distances $d(\underline{x}_i, \underline{x}_j)$ closely match dissimilarities.

Measure of goodness-of-fit: Stress

- ▶ Let the new configuration be obtained by projecting $\underline{x}_1, \dots, \underline{x}_n$ onto a k -D subspace.
- ▶ Minimizing “lack of fit” between the two configurations (dissimilarities and distances)

Stress of the configuration: $\Phi(\Delta, D) = \sum (\delta_{ij}^2 - d_{ij}^2)$

- ▶ Then, the subspace is spanned by the k largest principal components.

Classical Solution: Metric method

Given: $n \times n$ Δ dissimilarities matrix of n points in m -D space

Questions:

- Is Δ an interpoint distance matrix in Euclidean space?
- If yes, what is the dimension?
- What are the coordinates?

First, check whether Δ is Euclidean:

define

$A_{n \times n}$ with elements $a_{ij} = -(1/2)\delta_{ij}^2$

$B = HAH = U\Lambda U^T$ “centered inner product matrix”

$H_{n \times n} = I - n^{-1}\underline{1}\underline{1}^T$ “centering matrix”

→ Δ is Euclidean if and only if B is positive semidefinite

i.e. $\lambda(B) \geq 0$, with $\text{rank} \leq m$

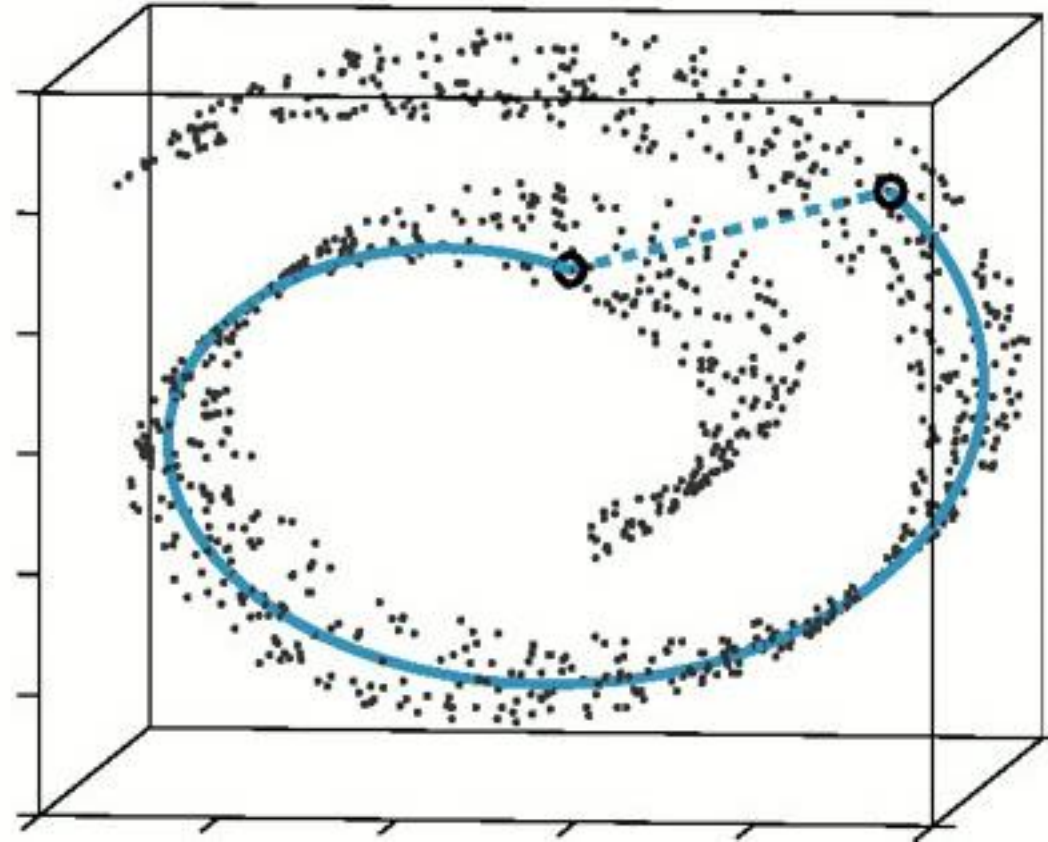
Classical Solution: Metric method

If Δ is Euclidean:

- Construct the matrix A
- Obtain the matrix B with elements $b_{ij} = a_{ij} - a_{i.} - a_{.j} + a_{..}$
- Find the k largest eigenvalues $\lambda_1 > \dots > \lambda_k$ of B , with corresponding eigenvectors $X = (\underline{x}_1, \dots, \underline{x}_k)$ which are normalized by $\underline{x}_i^T \underline{x}_i = \lambda_i$, $i = 1, \dots, k$ ($k < m$, is whatever we pick)
- Then, the required coordinates of the reconstruction points P_i are $\underline{x}_i = (x_{i1}, \dots, x_{in})^T$

The “Swiss roll” data set

A



Unlike the geodesic distance, the Euclidean distance cannot reflect the geometric structure of the data points

Approximating Geodesic Distances

Neighboring points: input-space distance

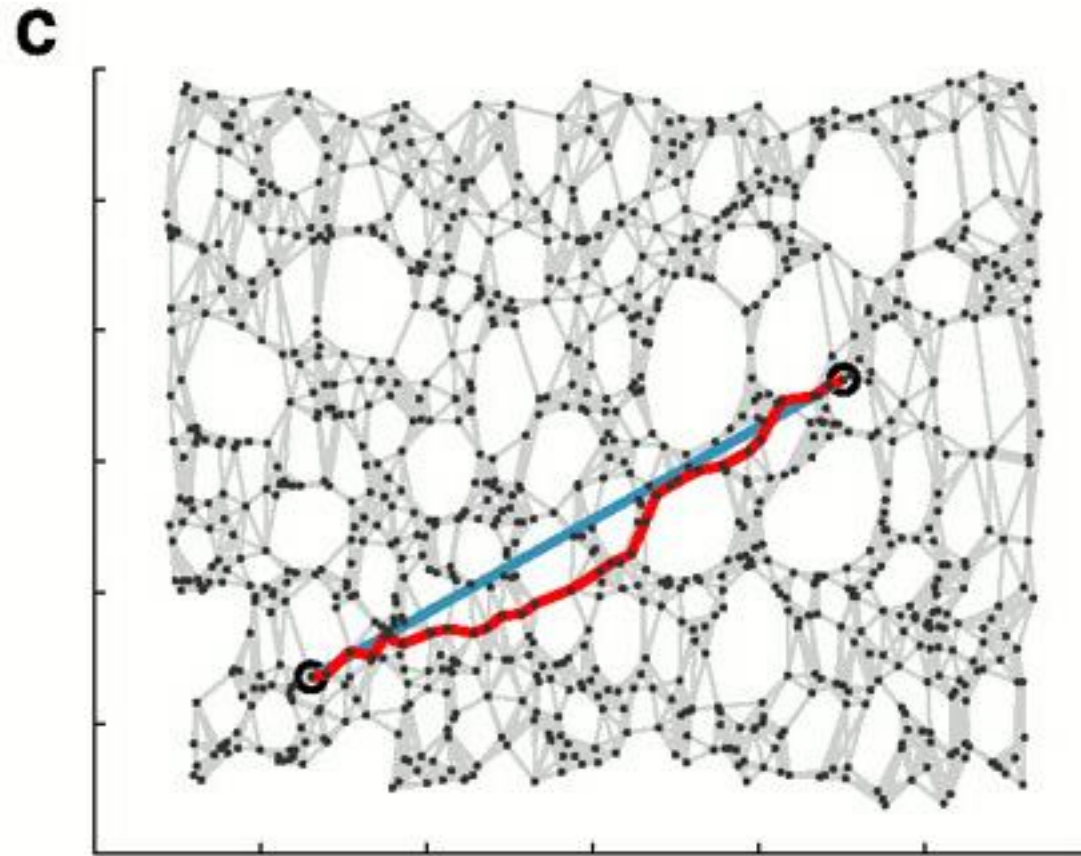
Faraway points: a sequence of “short hops” between neighboring points

Method: Finding shortest paths in a graph with edges connecting neighboring data points

Isomap Algorithm

Step	Name	Description
1 $O(DN^2)$	Construct neighborhood graph, G	Compute matrix $\mathbf{D}_G = \{d_x(i,j)\}$ $d_x(i,j)$ = Euclidean distance between neighbors
2 $O(DN^2)$	Compute shortest paths between <i>all</i> pairs	Compute matrix $\mathbf{D}_G = \{d_G(i,j)\}$ $d_G(i,j)$ = sequence of hops = approx geodesic dist.
3 $O(dN^2)$	Construct k-dimensional coordinate vectors	Apply MDS to $\mathbf{D}_G$ instead of $\mathbf{D}_x$

Isomap Output



The 2-D embedding recovered by Isomap

Summary on Isomap Algorithm

Advantages:

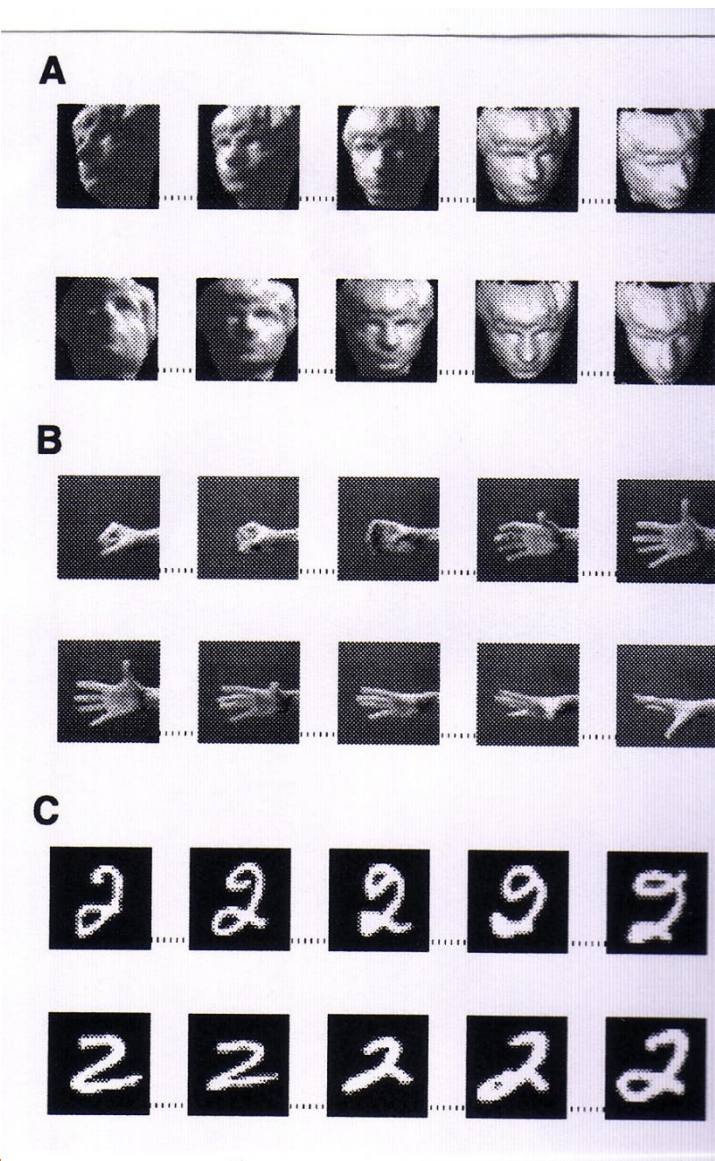
- ▶ Nonlinear
- ▶ Non-iterative
- ▶ Globally optimal
- ▶ Parameters: k or ε (chosen fixed radius)

Disadvantages:

- ▶ Graph discreteness overestimates the geodesic distance
- ▶ k must be high to avoid “linear shortcuts” near regions of high surface curvature

Using Isomap to discover
the intrinsic manifold in a
set of face images





Linear methods cannot interpolate properly between the leftmost and rightmost images in each row.

This is because the interpolated images are NOT averages of the images at the two ends.

Isomap does not interpolate properly either because it can only use examples from the training set. It cannot create new images.

But it is better than linear methods.

Maps that preserve local geometry

- ▶ The idea is to make the local configurations of points in the low-dimensional space resemble the local configurations in the high-dimensional space.
- ▶ We need a coordinate-free way of representing a local configuration.
- ▶ If we represent a point as a weighted average of nearby points, the weights describe the local configuration.

$$\mathbf{x}_i \approx \sum_j w_{ij} \mathbf{x}_j$$

Finding the optimal weights

- ▶ This is easy.
- ▶ Minimize the squared “construction” errors subject to the sum of the weights being 1.

$$Cost = \sum_i \left\| \mathbf{x}_i - \sum_{j \in N(i)} w_{ij} \mathbf{x}_j \right\|^2, \quad \sum_{j \in N(i)} w_{ij} = 1$$

- ▶ If the construction is done using less neighbors than the dimensionality of $\mathbf{x}$, there will generally be some construction error
- ▶ The error will be small if there are as many neighbors as the dimensionality of the underlying noisy manifold.

A sensible but inefficient way to use the local weights

- ▶ Assume a low-dimensional latent space.
 - ▶ Each datapoint $\mathbf{x}_i$ has latent coordinates $\mathbf{y}_i$.
- ▶ Find a set of latent points that minimize the construction errors produced by a two-stage process:
 - ▶ 1. First use the latent points to compute the local weights that construct $\mathbf{y}_i$ from its neighbors.
 - ▶ 2. Use those weights to construct the high-dimensional coordinates of a datapoint $\mathbf{x}_i$ from the high-dimensional coordinates of its neighbors.
- ▶ Unfortunately, this is a hard optimization problem.
 - ▶ Iterative solutions are expensive because they must repeatedly measure the construction error in the high-dimensional space.

Local Linear Embedding:

- ▶ Instead of using the the latent points plus the other datapoints to construct each held-out datapoint, do it the other way around.
- ▶ Use the datapoints to determine the local weights, then try to construct each latent point from its neighbors.
 - ▶ Now the construction error is in the low-dimensional latent space.
- ▶ We only use the high-dimensional space once to get the local weights.
 - ▶ The local weights stay fixed during the optimization of the latent coordinates.
 - ▶ This is a much easier search.

The convex optimization

fixed weights
↓

$$Cost = \sum_i \left\| \mathbf{y}_i - \sum_{j \in N(i)} w_{ij} \mathbf{y}_j \right\|^2$$

- ▶ Find the $\mathbf{y}$'s that minimize the cost subject to the constraint that the $\mathbf{y}$'s have unit variance on each dimension.
- ▶ Why do we need to impose a constraint on the variance?

The collapse problem

- ▶ If all of the latent points are identical, we can construct each of them perfectly as a weighted average of its neighbors.
 - ▶ The root cause of this problem is that we are optimizing the wrong thing.
 - ▶ But maybe we can fix things up by adding a constraint that prevents collapse.
- ▶ Insist that the latent points have unit variance on each latent dimension.
 - ▶ This helps a lot, but sometimes LLE can satisfy this constraint without doing what we really intend.

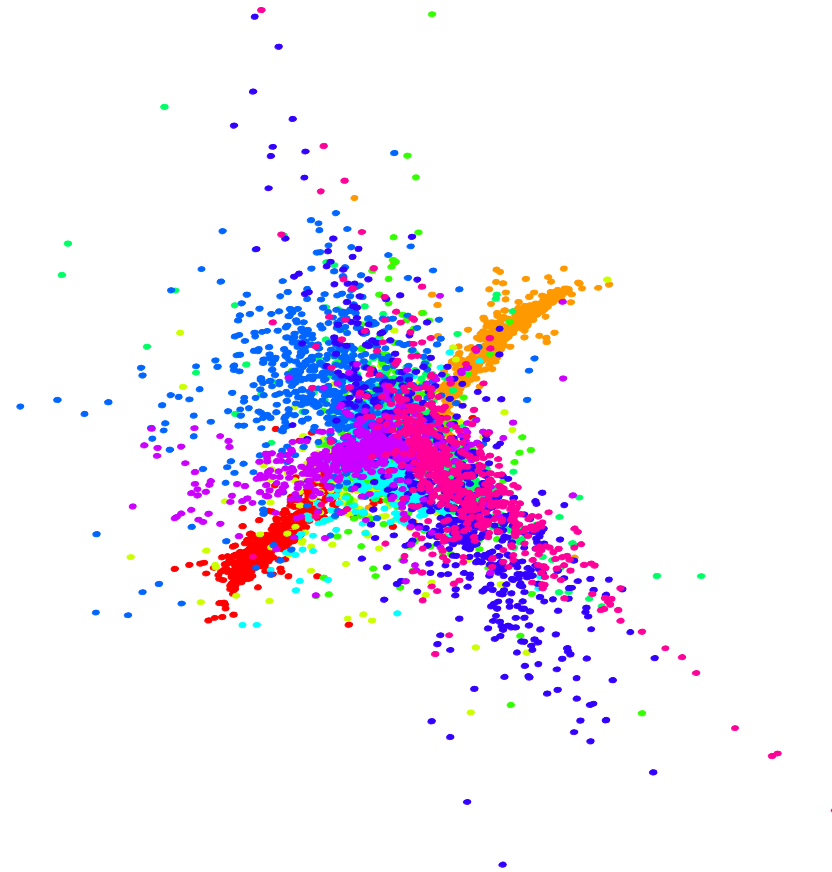
Failure modes of LLE

- ▶ If the neighborhood graph has several disconnected pieces, we can satisfy the unit variance constraint and still have collapses.
- ▶ Even if the graph is fully connected, it may be possible to collapse all the densely connected regions and satisfy the variance constraint by paying a high cost for a few outliers.



A typical embedding found by LLE

- ▶ LLE embeddings often look like this.
- ▶ Most of the data is close to the center of the space.
- ▶ A few points are far from the center to satisfy the unit variance constraint.



A comment on LLE

- ▶ It has two very attractive features
 - ▶ 1. The only free parameters are the dimensionality of the latent space and the number of neighbors that are used to determine the local weights.
 - ▶ 2. The optimization is convex so we don't need multiple tries and we don't need to fiddle with optimization parameters.
- ▶ It has one bad feature:
 - ▶ It is not optimizing the right thing!
 - ▶ One consequence is that it does not have any incentive to keep widely separated datapoints far apart in the low-dimensional map.

A probabilistic version of local MDS

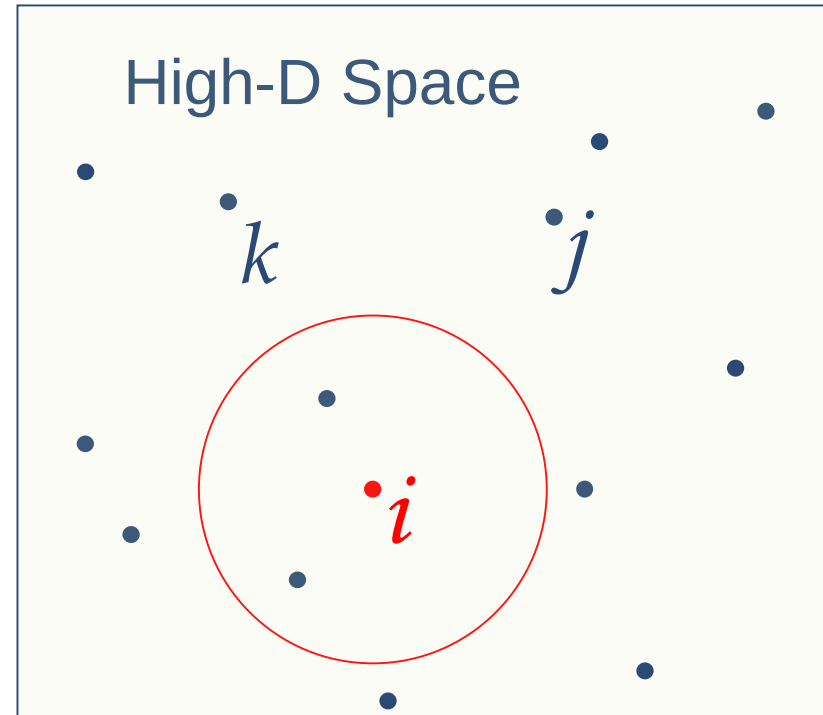
- ▶ It is more important to get local distances right than non-local ones, but getting infinitesimal distances right is not infinitely important.
- ▶ All the small distances are about equally important to model correctly.
- ▶ Stochastic neighbor embedding has a probabilistic way of deciding if a pairwise distance is “local”.

Stochastic Neighbor Embedding

- ▶ First convert each high-dimensional similarity into the probability that one data point will pick the other data point as its neighbor.
- ▶ To evaluate a map:
 - ▶ Use the pairwise distances in the low-dimensional map to define the probability that a map point will pick another map point as its neighbor.
 - ▶ Compute the Kullback-Leibler divergence between the probabilities in the high-dimensional and low-dimensional spaces.

A probabilistic local method

- ▶ Each point in high-D has a conditional probability of picking each other point as its neighbor.
- ▶ The distribution over neighbors is based on the high-D pairwise distances.
 - ▶ If we do not have coordinates for the datapoints we can use a matrix of dissimilarities instead of pairwise distances.



probability of picking j
given that you start at i

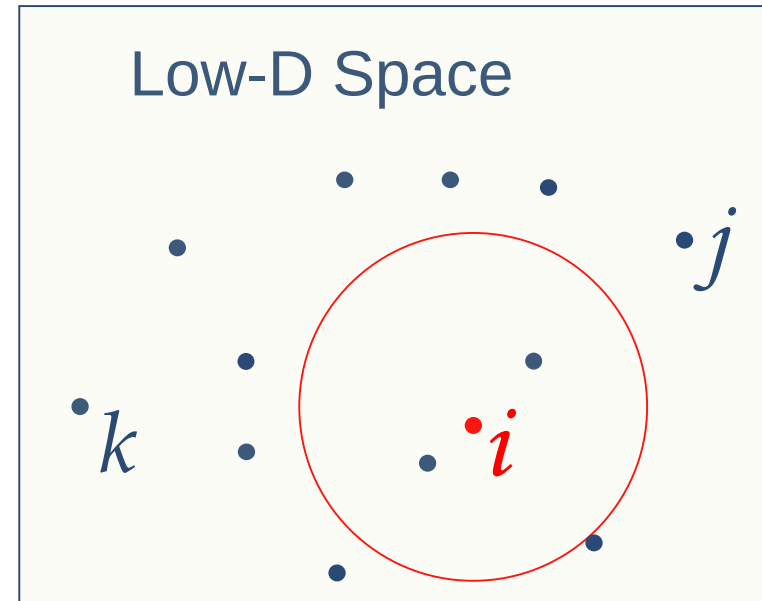
$$p_{j|i} = \frac{e^{-d_{ij}^2 / 2\sigma_i^2}}{\sum_k e^{-d_{ik}^2 / 2\sigma_i^2}}$$

Throwing away the raw data

- ▶ The probabilities that each point picks other points as its neighbor contains all of the information we are going to use for finding the manifold.
- ▶ Once we have the probabilities $p_{j|i}$ we do not need to do any more computations in the high-dimensional space.
- ▶ The input could be “dissimilarities” between pairs of datapoints instead of the locations of individual datapoints in a high-dimensional space.

Evaluating an arrangement of the data in a low-dimensional space

- ▶ Give each datapoint a location in the low- dimensional space.
- ▶ Evaluate this representation by seeing how well the low-D probabilities model the high-D ones.



probability of picking j
given that you start at i

$$q_{j|i} = \frac{e^{-d_{ij}^2}}{\sum_k e^{-d_{ik}^2}}$$

The cost function for a low-dimensional representation

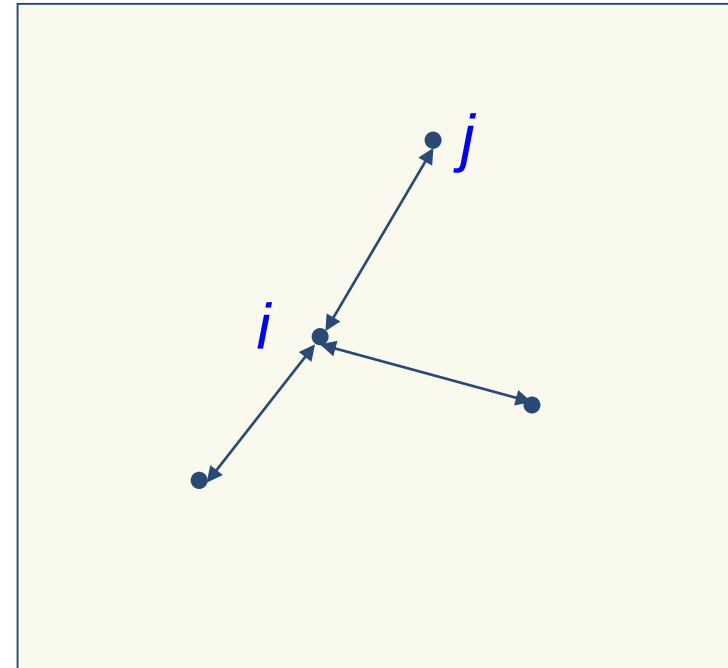
$$Cost = \sum_i KL(P_i \parallel Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$$

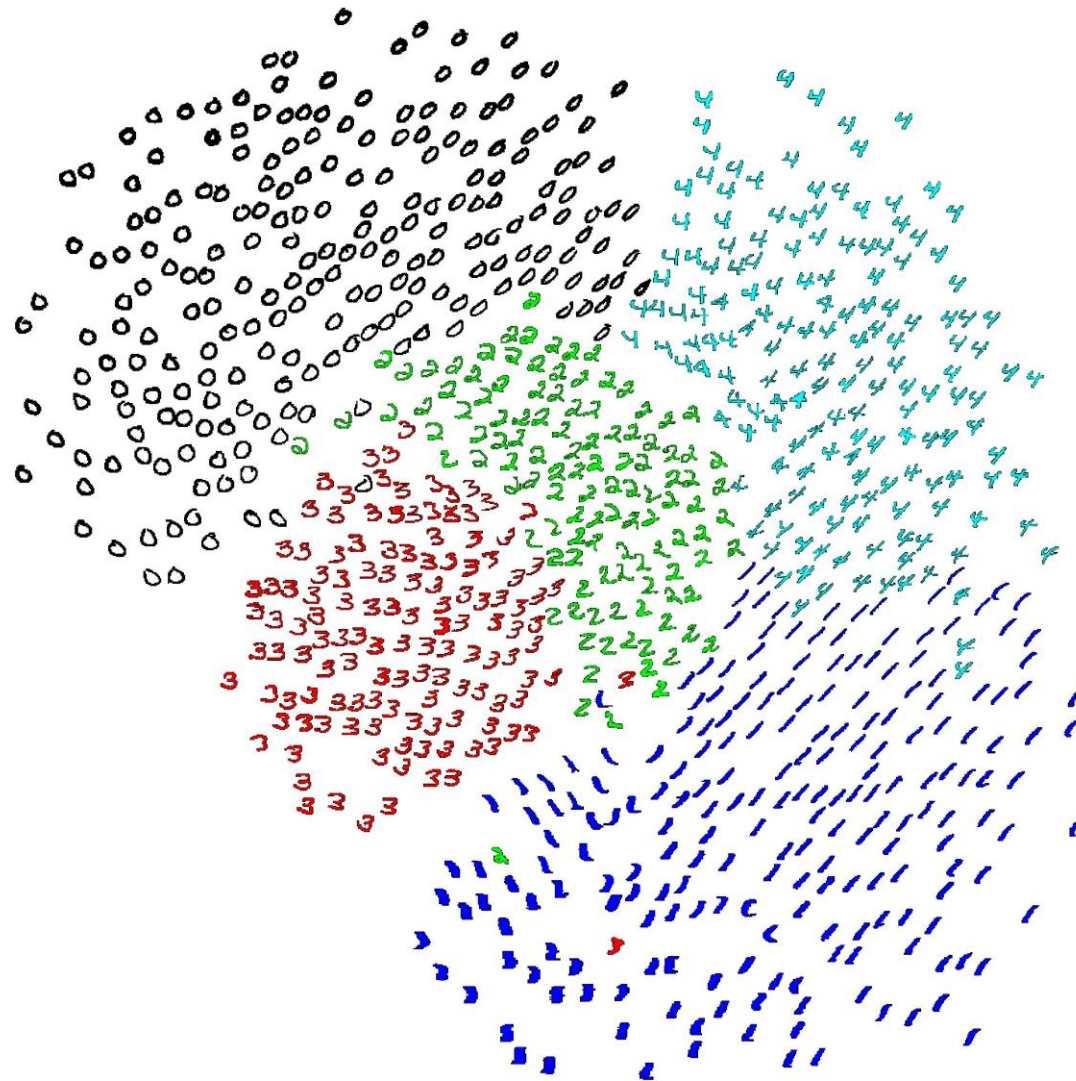
- ▶ For points where p_{ij} is large and q_{ij} is small we lose a lot.
 - ▶ Nearby points in high-D really want to be nearby in low-D
- ▶ For points where q_{ij} is large and p_{ij} is small we lose a little because we waste some of the probability mass in the Q_i distribution.
- ▶ Widely separated points in high-D have a mild preference for being widely separated in low-D.

The forces acting on the low-dimensional points

$$\frac{\partial \text{Cost}}{\partial \mathbf{y}_i} = 2 \sum_j (\mathbf{y}_j - \mathbf{y}_i) (p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j})$$

- Points are pulled towards each other if the p's are bigger than the q's and repelled if the q's are bigger than the p's





Unsupervised
SNE embedding
of the digits 0-4.
Not all the data
is displayed

Picking the radius of the Gaussian that is used to compute the p 's

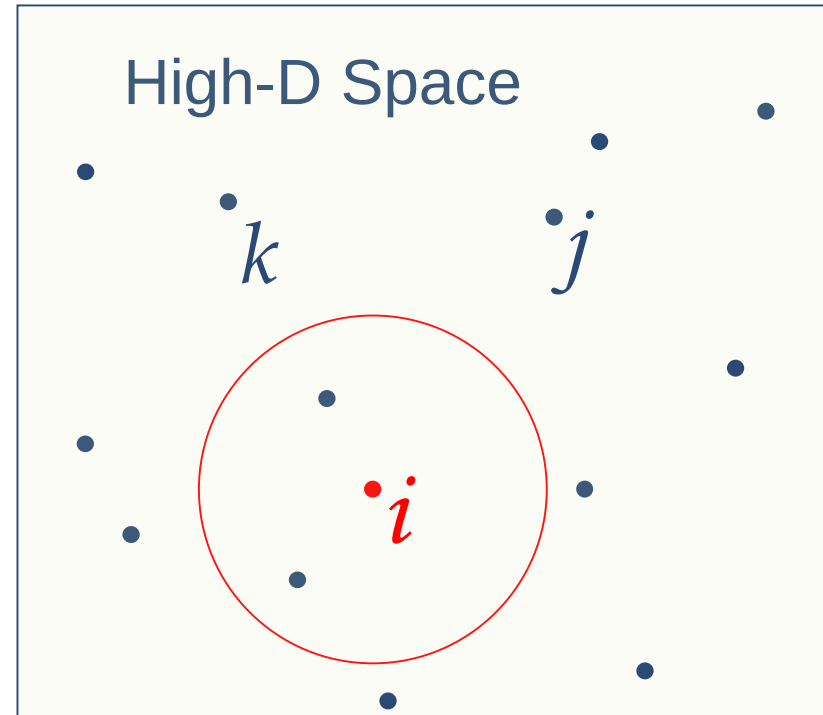
- ▶ We need to use different radii in different parts of the space so that we keep the effective number of neighbors about constant.
- ▶ A big radius leads to a high entropy for the distribution over neighbors of i .
- ▶ A small radius leads to a low entropy.
- ▶ So decide what entropy you want and then find the radius that produces that entropy.
- ▶ Its easier to specify 2^{entropy}
 - ▶ This is called the perplexity
 - ▶ It is the effective number of neighbors.

Symmetric SNE

- ▶ There is a simpler version of SNE which seems to work about equally well.
- ▶ Symmetric SNE works best if we use different procedures for computing the p 's and the q 's
 - ▶ This destroys the nice property that if we embed in a space of the same dimension as the data, the data itself is the optimal solution.

Computing the p's for symmetric SNE

- ▶ Each high dimensional point, i , has a conditional probability of picking each other point, j , as its neighbor.
- ▶ The conditional distribution over neighbors is based on the high-dimensional pairwise distances.



$$p_{j|i} = \frac{e^{-d_{ij}^2 / 2\sigma_i^2}}{\sum_k e^{-d_{ik}^2 / 2\sigma_i^2}}$$

probability of picking j
given that you start at i

Turning conditional probabilities into pairwise probabilities

To get a symmetric probability between i and j we sum the two conditional probabilities and divide by the number of points (points are not allowed to choose themselves).

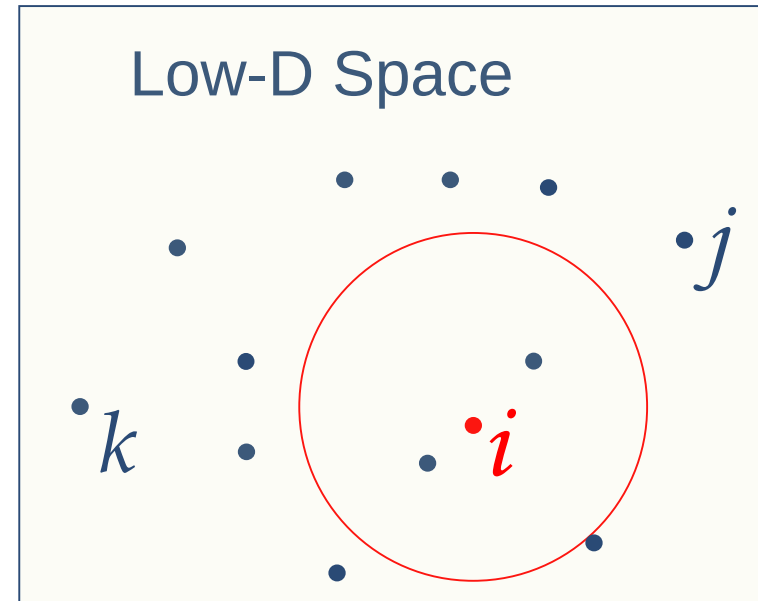
joint probability of
picking the pair i, j $\longrightarrow$ $p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$

This ensures that all the pairwise probabilities sum to 1 so they can be treated as probabilities.

$$\sum_{i,j} p_{ij} = 1$$

Evaluating an arrangement of the points in the low-dimensional space

- ▶ Give each data-point a location in the low- dimensional space.
- ▶ Define low-dimensional probabilities symmetrically.
- ▶ Evaluate the representation by seeing how well the low-D probabilities model the high-D affinities.



$$q_{ij} = \frac{e^{-d_{ij}^2}}{\sum_{k < l} e^{-d_{kl}^2}}$$

The cost function for a low-dimensional representation

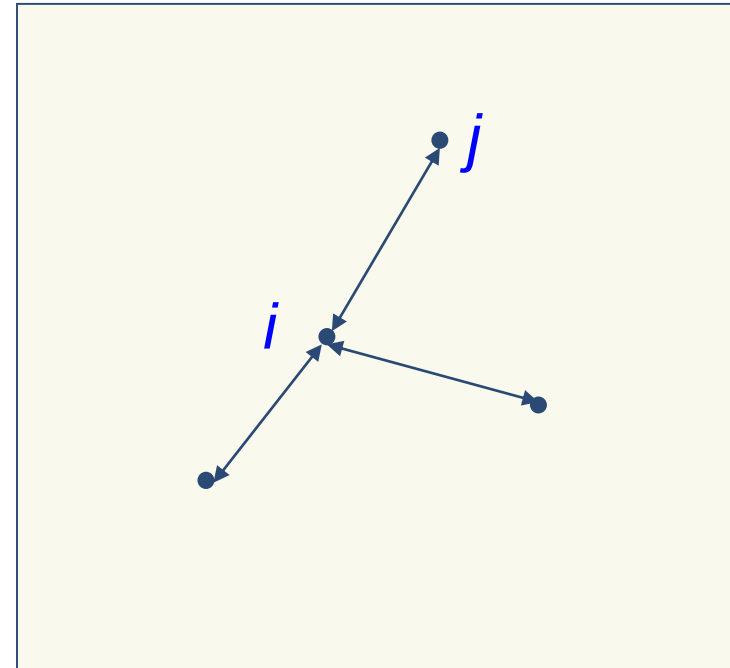
$$Cost = KL(P \parallel Q) = \sum_{i < j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

- It's a single KL instead of the sum of one KL for each datapoint.

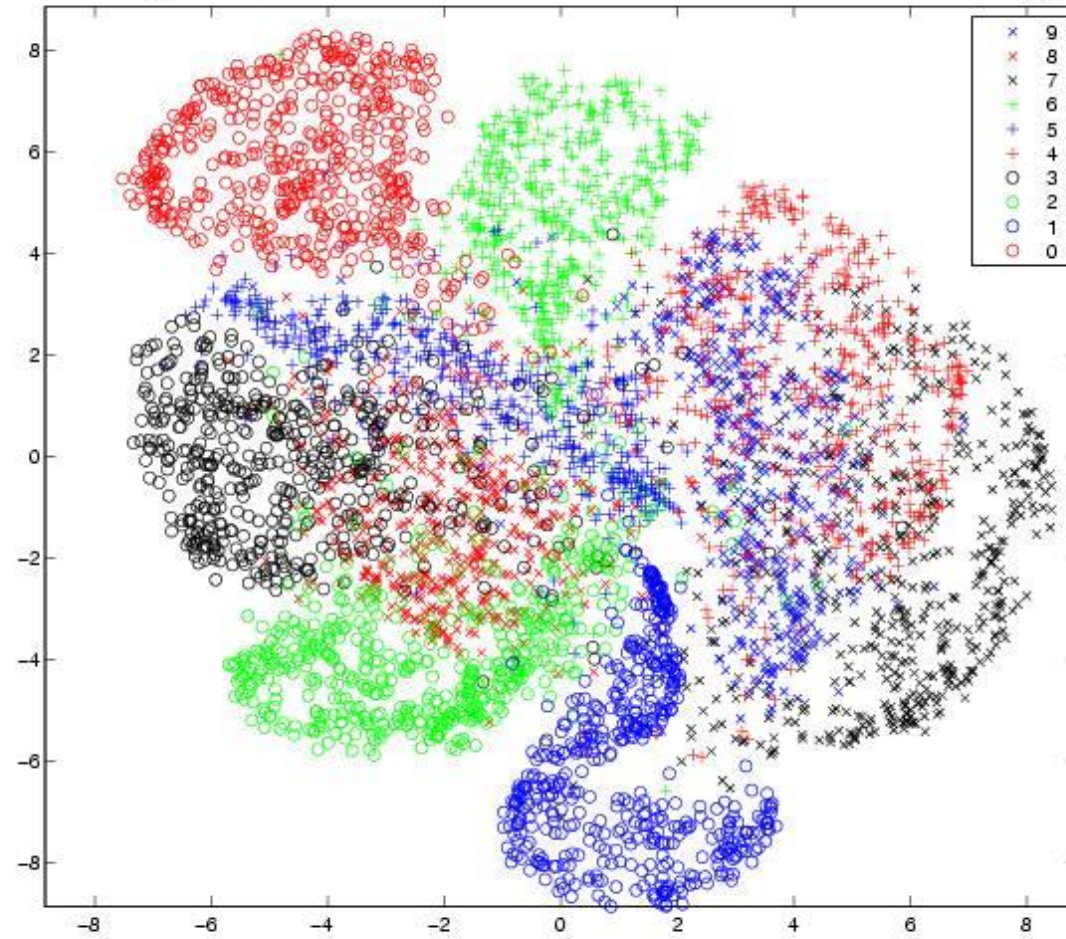
The forces acting on the low-dimensional points

$$\frac{\partial KL(P \parallel Q)}{\partial \mathbf{y}_i} = 2 \sum_j (\overset{\text{extension}}{\downarrow} \mathbf{y}_i - \mathbf{y}_j) (\overset{\text{stiffness}}{\downarrow} p_{ij} - q_{ij})$$

- Points are pulled towards each other if the p's are bigger than the q's and repelled if the q's are bigger than the p's
- Its equivalent to having springs whose stiffnesses are set dynamically.



SNE applied to 30-dimensional PCA codes of 5000 MNIST digits



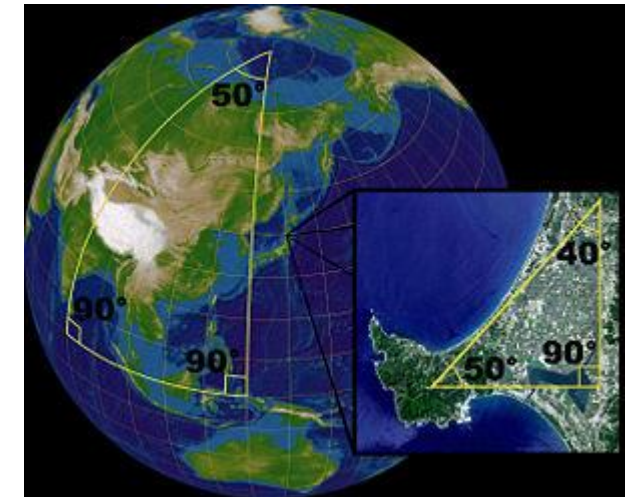
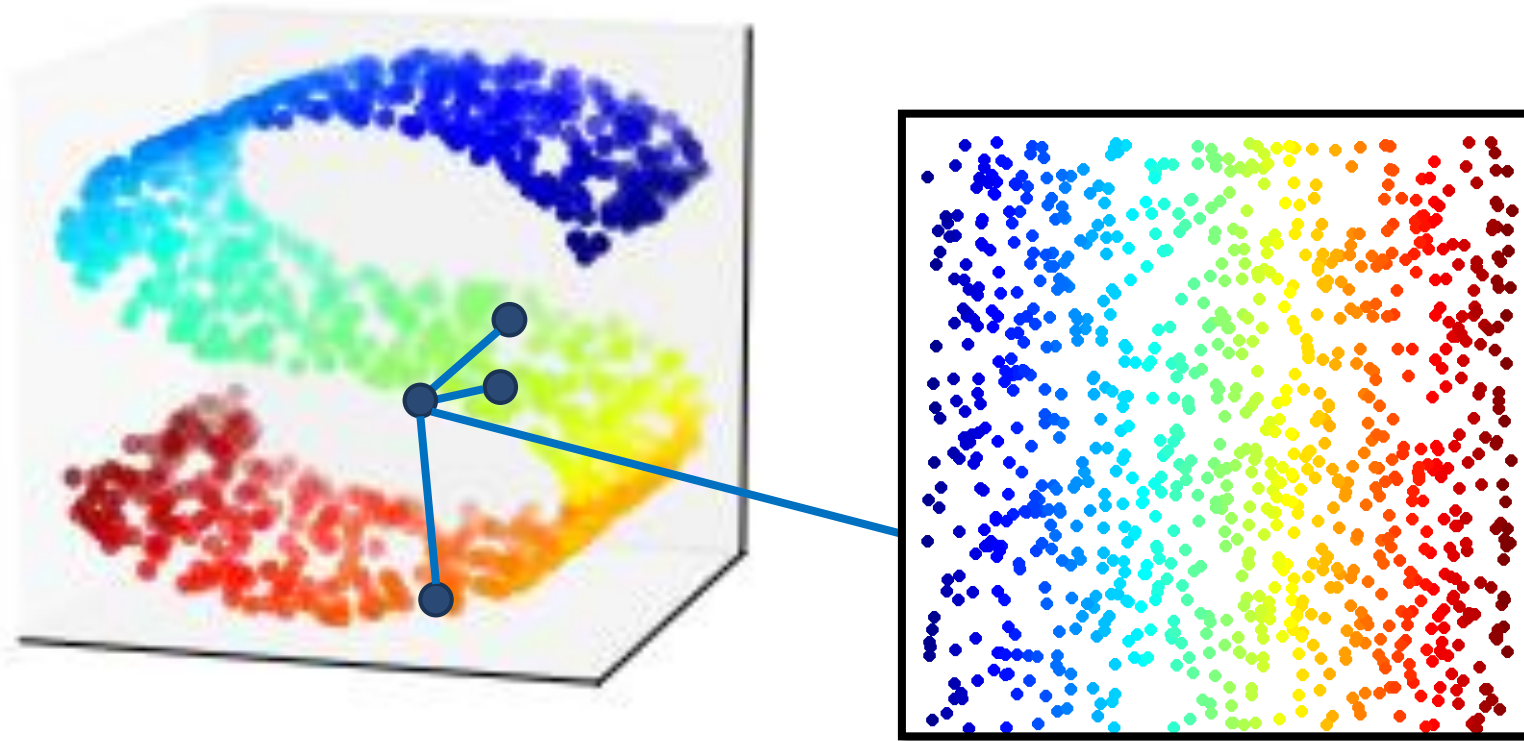
Optimization methods for SNE

- ▶ We get much better global organization if we use annealing.
 - ▶ Add Gaussian noise to the y locations after each update.
 - ▶ Reduce the amount of noise on each iteration.
 - ▶ Spend a long time at the noise level at which the global structure starts to form from the hot plasma of map points.
- ▶ It also helps to use momentum (especially at the end).
- ▶ It helps to use an adaptive global step-size.

Optimization hacks

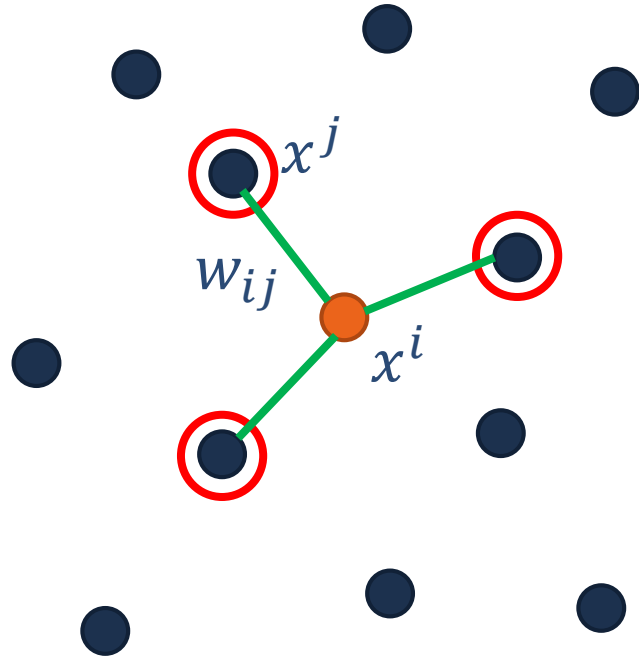
- ▶ **Reputable hack:** Introduce a penalty term that keeps all the map-points close together.
 - ▶ Then gradually relax the penalty to break symmetry slowly.
- ▶ **Disreputable hack:** Allow the probabilities to add up to 4.
 - ▶ This causes the map-points to curdle into small clusters leaving lots of space for clusters to move past each other.
 - ▶ Then make the probabilities add up to 1.

t-SNE



Manifold Learning Suitable for clustering or following supervised learning

Locally Linear Embedding (LLE)



w_{ij} represents the relation
between x^i and x^j

Find a set of w_{ij} minimizing

$$\sum_i \left\| x^i - \sum_j w_{ij} x^j \right\|_2$$

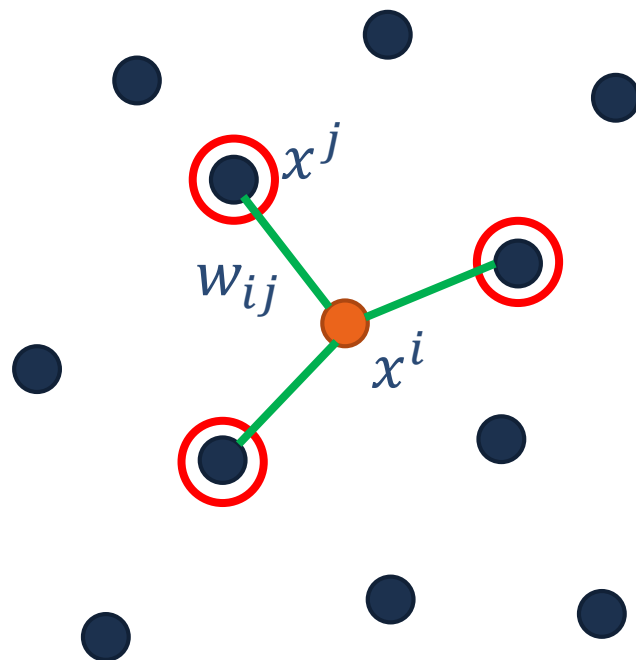
Then find the dimension reduction
results z^i and z^j based on w_{ij}

LLE

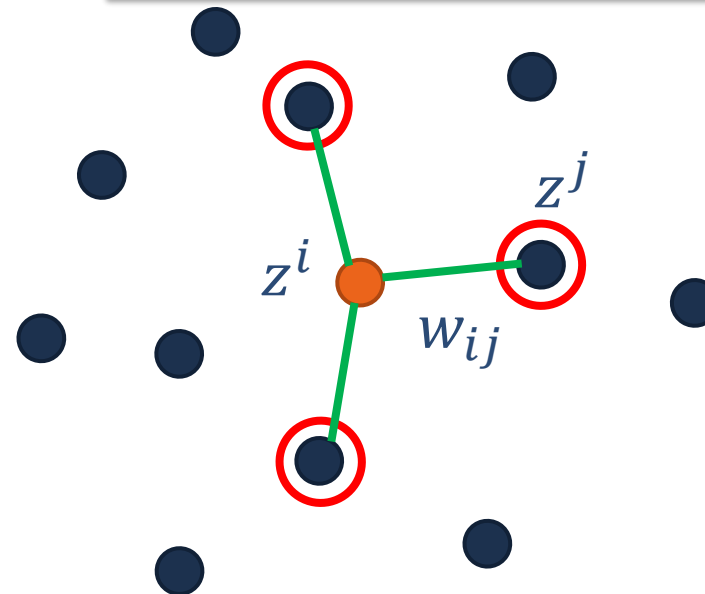
Find a set of z^i minimizing

$$\sum_i \left\| z^i - \sum_j w_{ij} z^j \right\|_2$$

Keep w_{ij} unchanged



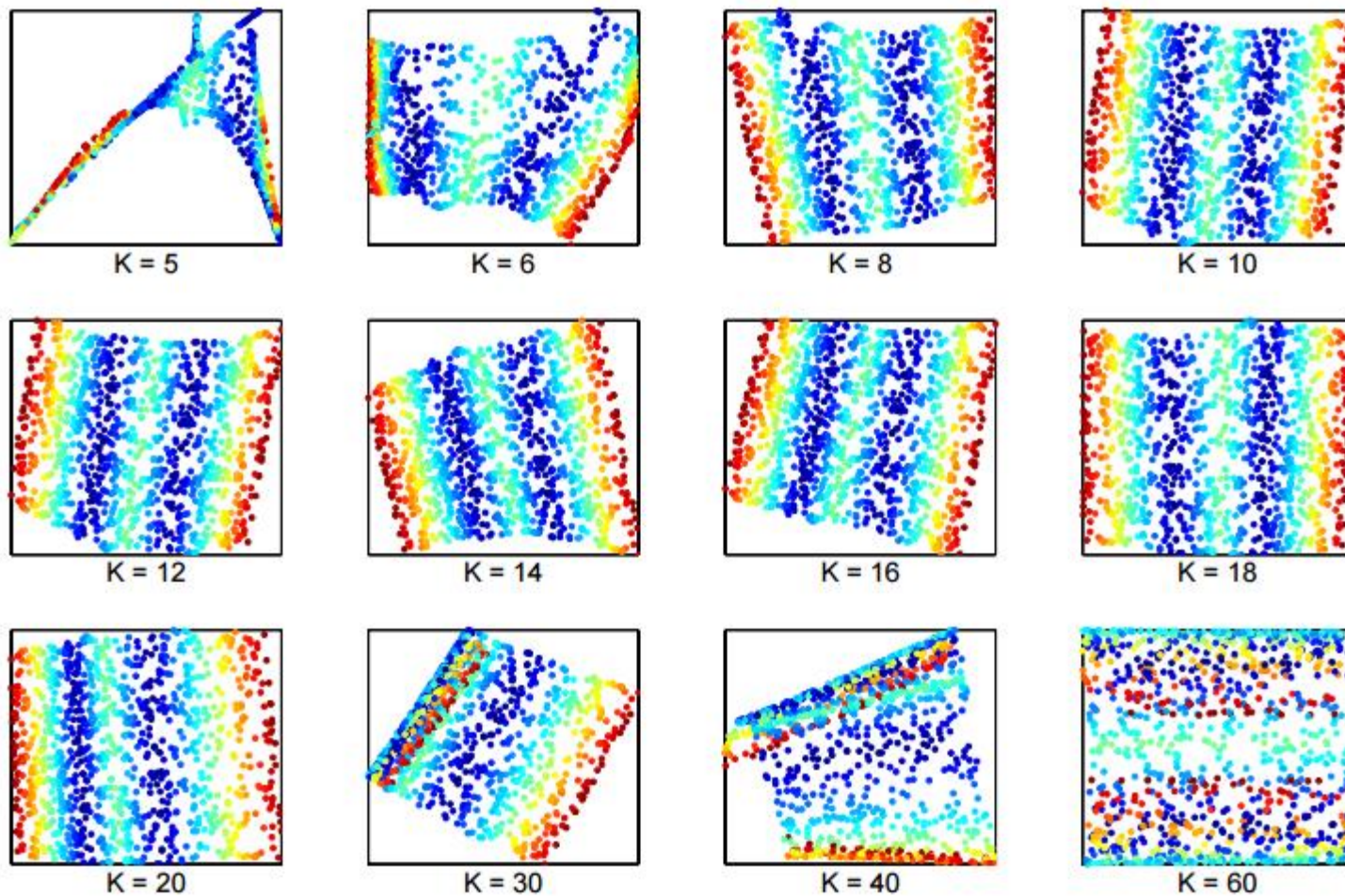
Original Space



New (Low-dim) Space

LLE

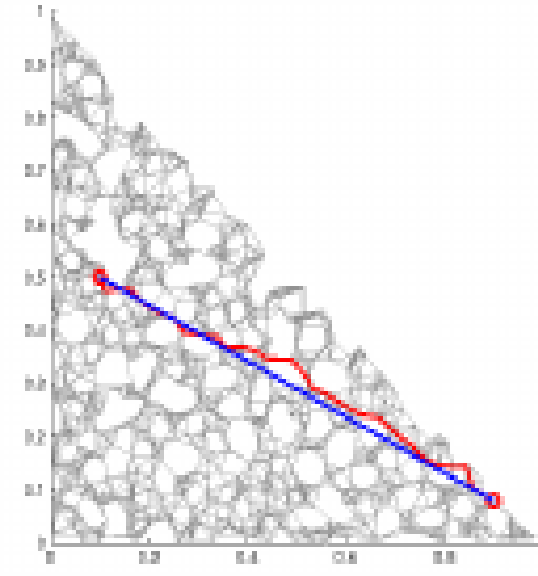
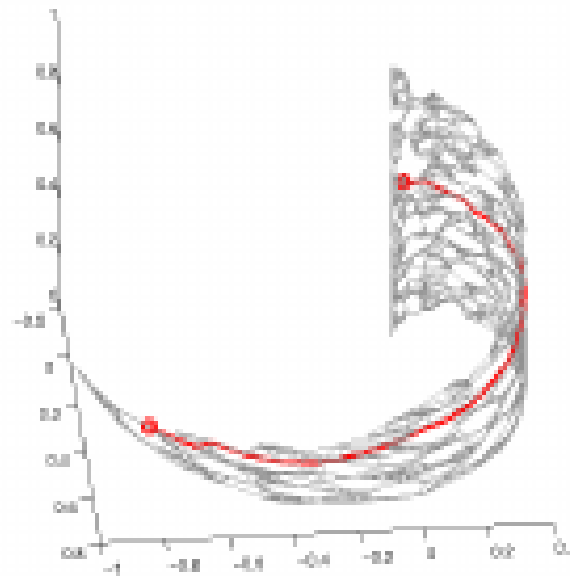
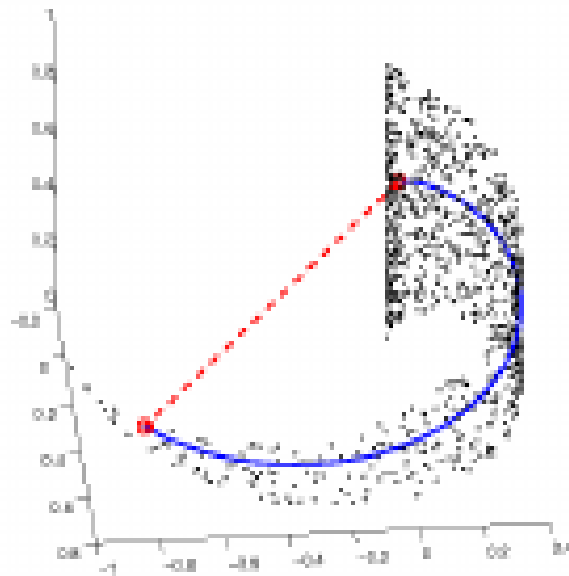
Lawrence K. Saul, Sam T. Roweis, “Think Globally, Fit Locally:
Unsupervised Learning of Low Dimensional Manifolds”, JMLR,
2013



Laplacian Eigenmaps

► Graph-based approach

Distance defined by graph
approximate the distance
on manifold



Construct the data points as
a graph

Laplacian Eigenmaps

- *Review in semi-supervised learning:* If x^1 and x^2 are close in a high density region, $\hat{y}^1$ and $\hat{y}^2$ are probably the same.

$$w_{i,j} = \begin{cases} \text{similarity} & \text{If connected} \\ 0 & \text{otherwise} \end{cases}$$

$$L = \sum_{x^r} C(y^r, \hat{y}^r) + \lambda S$$

As a regularization term

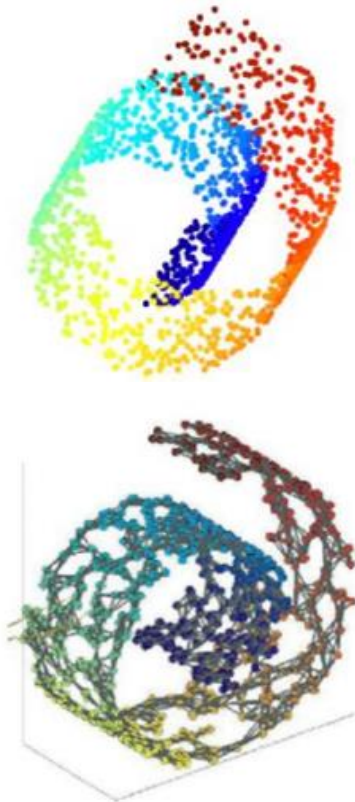
$$S = \frac{1}{2} \sum_{i,j} w_{i,j} (y^i - y^j)^2 = \mathbf{y}^T L \mathbf{y}$$

S evaluates how smooth your label is

L: $(R+U) \times (R+U)$ matrix

Graph Laplacian

$$L = D - W$$



Laplacian Eigenmaps

- *Dimension Reduction*: If x^1 and x^2 are close in a high density region, z^1 and z^2 are close to each other.

$$S = \frac{1}{2} \sum_{i,j} w_{i,j} (z^i - z^j)^2$$

Any How about $z^i = z^j = \mathbf{0}$?

problem?
Giving some constraints to

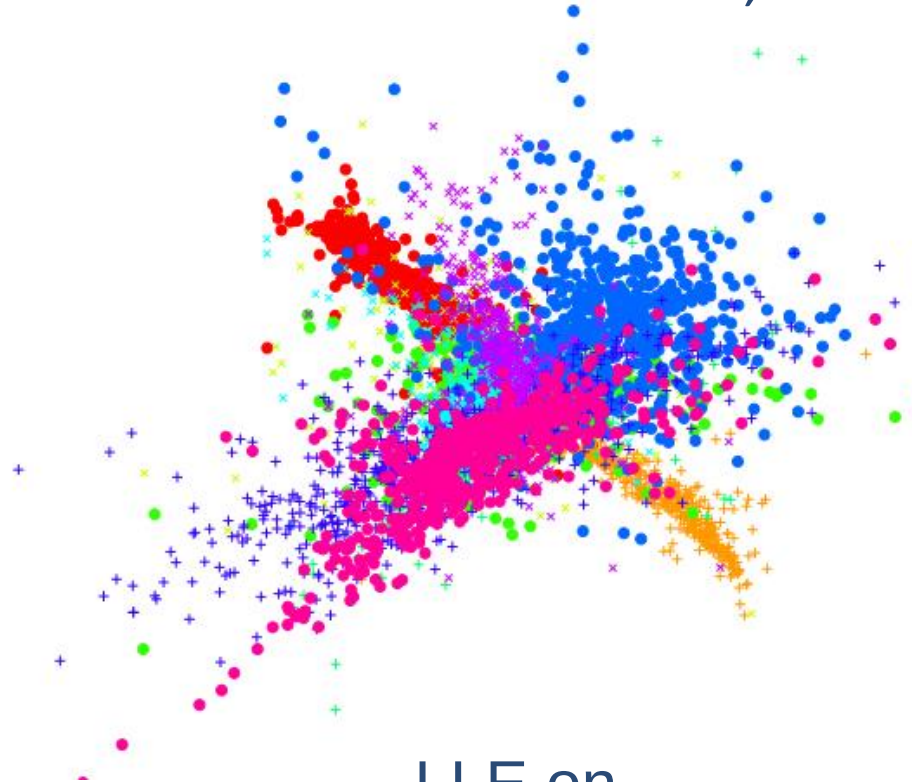
z:
If the dim of z is M, $\text{Span}\{z^1, z^2, \dots, z^N\} = \mathbb{R}^M$

Spectral clustering: clustering on z

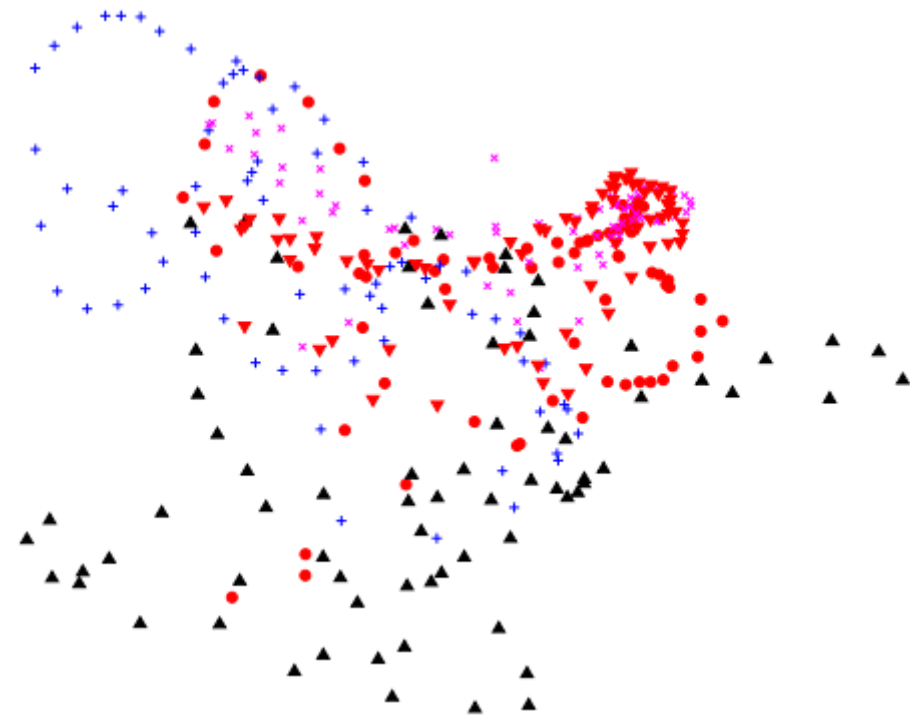
Belkin, M., Niyogi, P. Laplacian eigenmaps and spectral techniques for embedding and clustering. *Advances in neural information processing systems* . 2002

T-distributed Stochastic Neighbor Embedding (t-SNE)

- ▶ Problem of the previous approaches
 - ▶ Similar data are close, but different data may collapse



LLE on
MNIST



LLE on COIL-
20

t-SNE



Compute similarity between all pairs of x: $S(x^i, x^j)$

$$P(x^j | x^i) = \frac{S(x^i, x^j)}{\sum_{k \neq i} S(x^i, x^k)}$$

Compute similarity between all pairs of z: $S'(z^i, z^j)$

$$Q(z^j | z^i) = \frac{S'(z^i, z^j)}{\sum_{k \neq i} S'(z^i, z^k)}$$

Find a set of z making the two distributions as close as possible

$$\begin{aligned} L &= \sum_i KL(P(* | x^i) || Q(* | z^i)) \\ &= \sum_i \sum_j P(x^j | x^i) \log \frac{P(x^j | x^i)}{Q(z^j | z^i)} \end{aligned}$$

t-SNE – Similarity Measure

Ignore σ for simplicity

$$S(x^i, x^j)$$

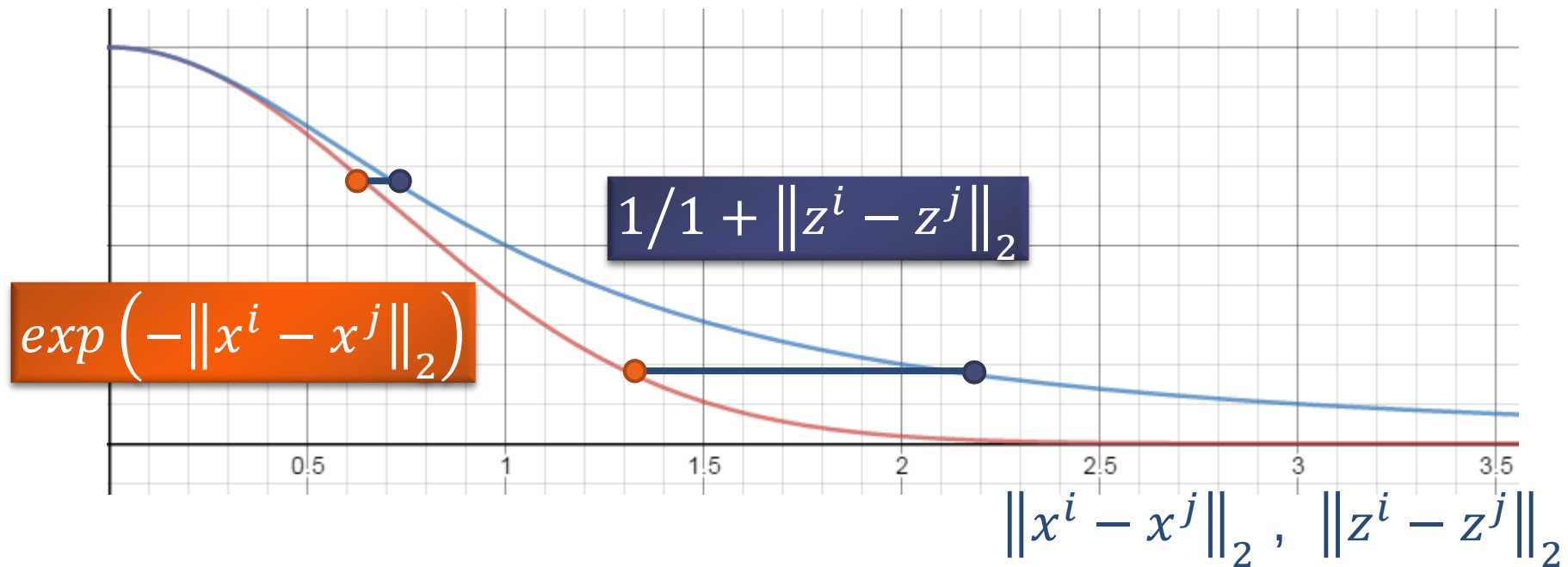
$$= \exp(-\|x^i - x^j\|_2)$$

SNE:

$$S'(z^i, z^j) = \exp(-\|z^i - z^j\|_2)$$

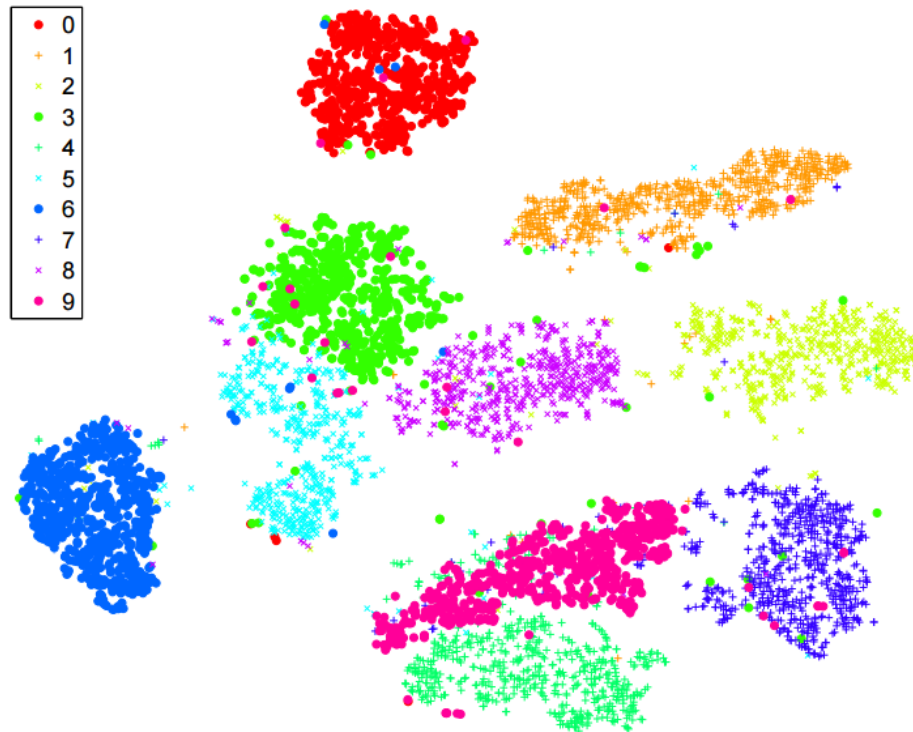
t-SNE:

$$S'(z^i, z^j) = 1 / (1 + \|z^i - z^j\|_2)$$



t-SNE

► Good at visualization

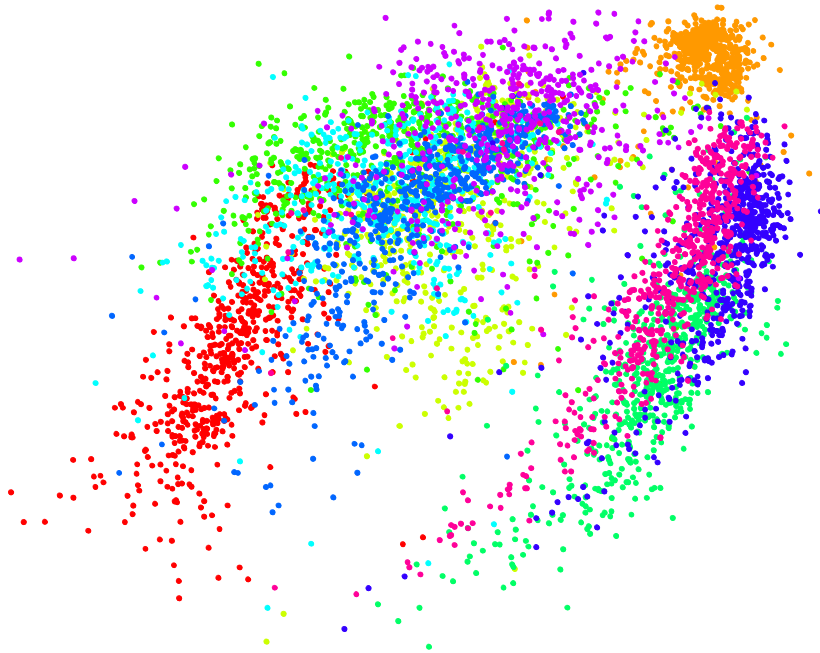


t-SNE on
MNIST

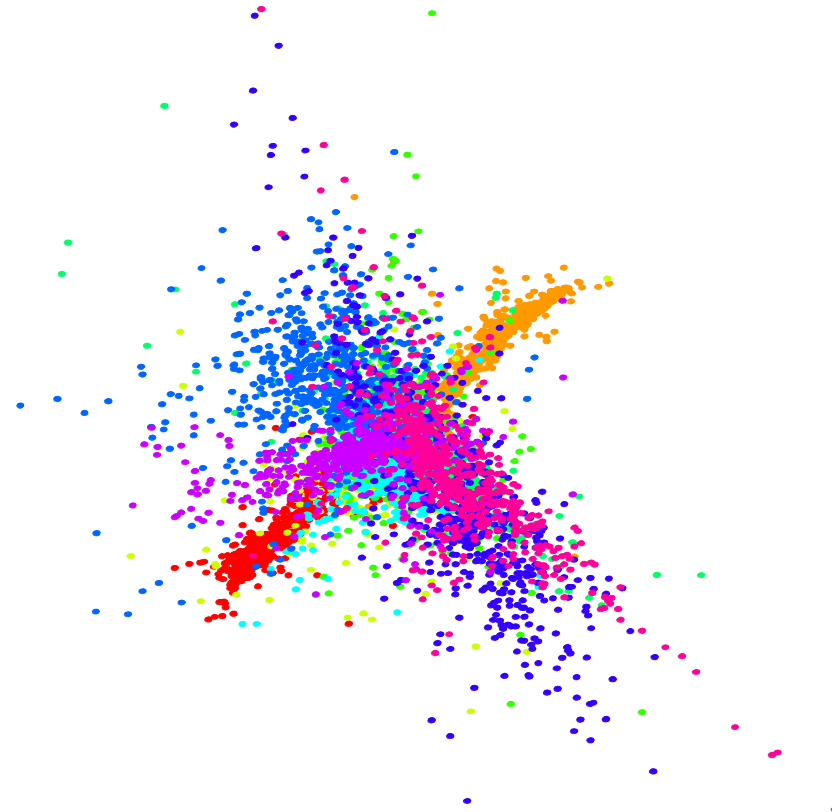


t-SNE on COIL-
20

Two other state-of-the-art dimensionality reduction methods on the 6000 MNIST digits

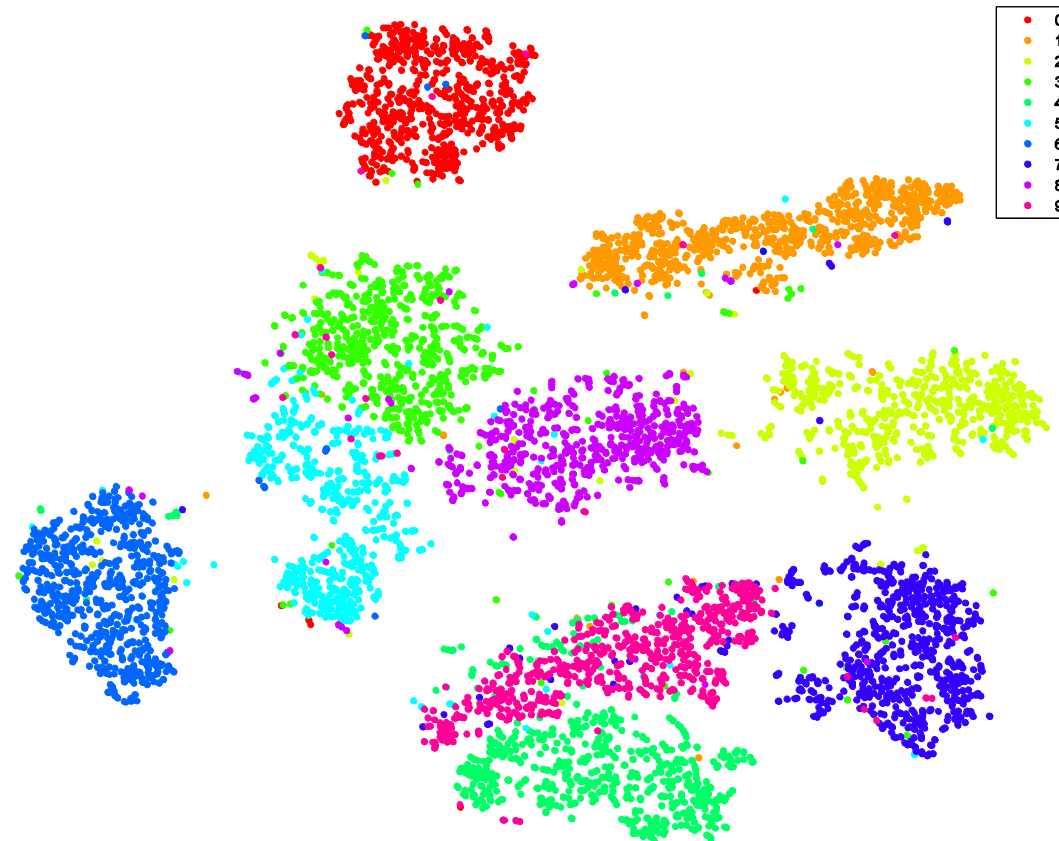


Isomap



Locally Linear Embedding

t-SNE on the 6000 MNIST digits

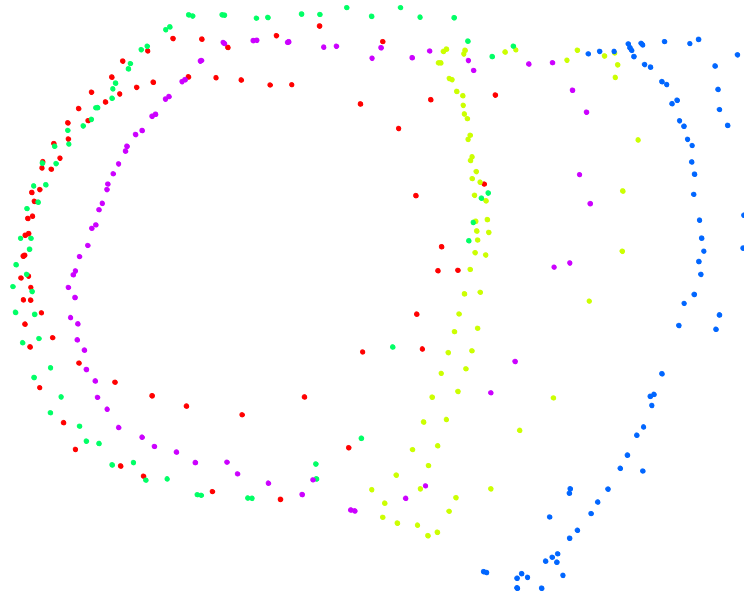


The COIL20 dataset

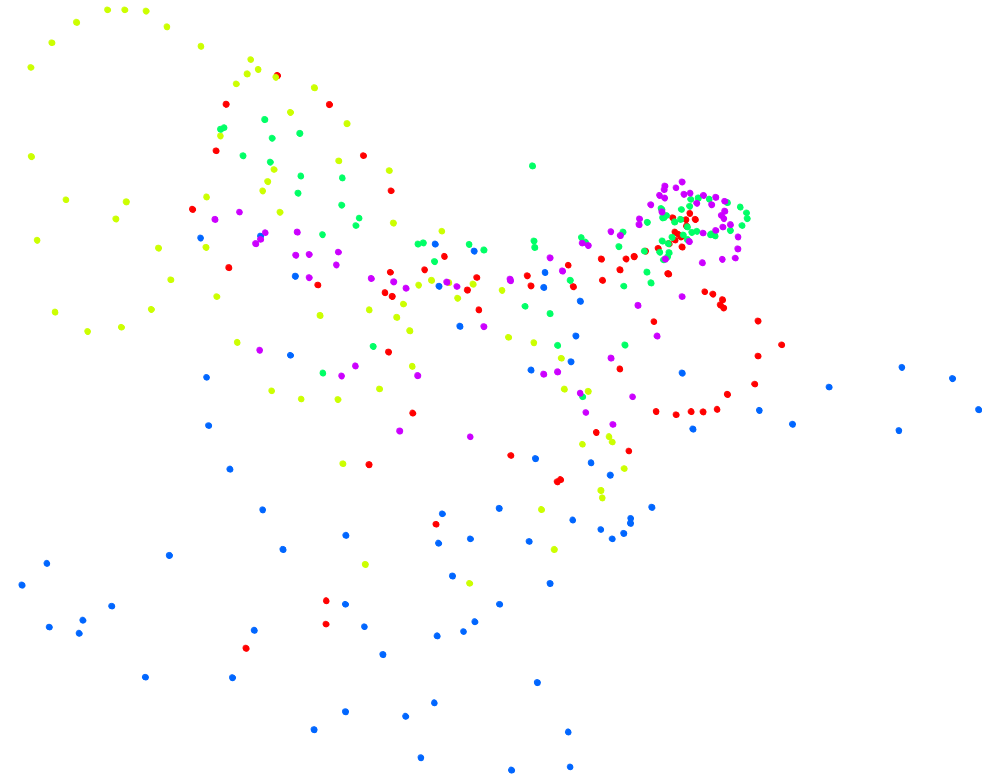


Each object is rotated about a vertical axis to produce a closed one-dimensional manifold of images.

Isomap & LLE for COIL20 dataset



Isomap



Locally Linear
Embedding

t-SNE for COIL20 dataset



Using t-SNE to see what you are thinking

